Numerical Double Pendulum Simulation

Stepan Katashynsky

May 11, 2026

1 Introduction

This report presents a numerical simulation approach for a nonlinear double pendulum system problem, implemented in Python and MATLAB. The system is modeled as a set of coupled ordinary differential equations of the first and second orders, and solved using the fourth-order Runge-Kutta method. The simulation includes real-time visualization of the pendulum motion, kinetic, potential and total energy analysis, phase portrait representation, and trajectory tracing of the second mass.

The double pendulum is one of the classical examples of a nonlinear mechanical systems. Although the system consists only of two connected pendulum arms, its motion can become highly complex and chaotic. Small changes in the initial conditions may lead to significantly differences in trajectories over time evolution. The purpose of the project is to demonstrate the application of numerical methods and dynamic system modeling. Two implementations were developed: one in Python and the other one in MATLAB. Both implementations use the same physical model, the same initial parameters, and the same numerical integration method.

2 Physical model

The double pendulum consists of two pendula connected in series. The first one is attached to a fixed pivot, while the second one is attached to the end of the first one.

The following assumptions are used:

- rods with lengths L_1 and L_2 are assumed to be massless,
- masses m_1 and m_2 are treated as the point masses,
- motion is restricted to a two-dimensional plane,
- energy loss is represented by linear damping coefficients γ_1 and γ_2,
- air resistance, rod flexibility, detailed joint friction, and three-dimensional effects are neglected.

The angles θ_1 and θ_2 define the angular positions of the two pendulum arms. The lengths of the rods are noted as L_1 and L_2, and the masses are noted as m_1 and m_2.

Table 1: Simulation parameters

Parameter	Description	Value
g	gravitational acceleration	$9.81\,\mathrm{m/s^2}$
m_1	mass of the first bob	$1\,\mathrm{kg}$
m_2	mass of the second bob	$1\,\mathrm{kg}$
L_1	length of the first rod	$1\,\mathrm{m}$
L_2	length of the second rod	$1\,\mathrm{m}$
γ_1	damping coefficient of the first pendulum	$0.02\,\mathrm{s^{-1}}$
γ_2	damping coefficient of the second pendulum	$0.08\,\mathrm{s^{-1}}$
Δt	time step	$0.01\,\mathrm{s}$

2.1 Coordinates definition

The Cartesian coordinates were chosen due to most handful use in this exact problem. These coordinate definitions allow the transformation of the angular motion of the system into Cartesian positions for visualization. Additional they are used to calculate kinetic and potential energies.

The coordinates of the first mass are defined as:

$$x_1 = L_1 \sin(\theta_1)$$

$$y_1 = -L_1 \cos(\theta_1).$$

The coordinates of the second mass are:

$$x_2 = x_1 + L_2 \sin(\theta_2)$$

$$y_2 = y_1 - L_2 \cos(\theta_2)$$

These coordinate definitions allow the angular motion of the system to be transformed into Cartesian positions for visualization. They are also used in the calculation of kinetic and potential energy.

2.2 Energy formulations

The squared velocity of the first mass is described as:

$$v_1^2 = (L_1\omega_1)^2$$

The squared velocity of the second mass is described as:

$$v_2^2 = v_1^2 + (L_2\omega_2)^2 + 2L_1L_2\omega_1\omega_2\cos(\theta_1 - \theta_2)$$

The kinetic energy of the system is described as:

$$T = \frac{1}{2}m_1v_1^2 + \frac{1}{2}m_2v_2^2$$

The potential energy is described as:

$$V = m_1 g y_1 + m_2 g y_2$$

Equivalently, using the angular coordinates, the potential energy can be rewritten as:

$$V = -L_1 g(m_1 + m_2)\cos(\theta_1) - L_2 m_2 g \cos(\theta_2)$$

The total mechanical energy is therefore:

$$E = T + V$$

In an udeal undamped system remain the total mechanical energy constant, therefore in this project damping is included, so the total mechanical energy is expected to decrease over time.

2.3 Lagrangian Formalism

The Lagrangian of the system is defined as:

$$\mathcal{L} = T - V$$

Using the kinetic and potential energy expressions, the Lagrangian becomes:

$$\mathcal{L} = \frac{1}{2}(m_1+m_2)L_1^2\dot{\theta}_1^2 + \frac{1}{2}m_2 L_2^2\dot{\theta}_2^2 + m_2 L_1 L_2 \dot{\theta}_1\dot{\theta}_2\cos(\theta_1-\theta_2) + (m_1+m_2)gL_1\cos(\theta_1) + m_2 g L_2\cos(\theta_2)$$

The equations of motion are obtained using the Euler-Lagrange equation

$$\frac{d}{dt}\left(\frac{\partial \mathcal{L}}{\partial \dot{q}_i}\right) - \frac{\partial \mathcal{L}}{\partial q_i} = 0$$

where q_i are represented as the generalized coordinates θ_1 and θ_2.

After applying the Euler-Lagrange equations, the double pendulum is described by two coupled nonlinear second-order differential equations. Following equations cannot be generally solved analytically for arbitrary initial conditions, so a numerical method is required.

2.4 State vector

For numerical integration, the second-order differential equations are rewritten as a first-order system.

The state vector is defined as:

$$y = \begin{bmatrix} \theta_1 \\ \omega_1 \\ \theta_2 \\ \omega_2 \end{bmatrix}$$

where

$$\omega_1 = \dot{\theta}_1 \qquad \omega_2 = \dot{\theta}_2$$

The derivative of the state vector is:

$$\frac{dy}{dt} = \begin{bmatrix} \dot{\theta}_1 \\ \dot{\omega}_1 \\ \dot{\theta}_2 \\ \dot{\omega}_2 \end{bmatrix} = \begin{bmatrix} \omega_1 \\ \alpha_1 \\ \omega_2 \\ \alpha_2 \end{bmatrix}$$

Here α_1 and α_2 are describing the angular accelerations of the first and second pendulum arms.

The derivative function does not compute the full trajectory directly. Instead, it computes the rate of change of the current state. The numerical integration method then uses this information in order to update the state over time.

2.5 Equations of motion used in the simulation

The following auxiliary variables are introduced in order to simplify the calculations:

$$\delta = \theta_1 - \theta_2$$

$$M = m_1 + m_2$$
$$\alpha = m_1 + m_2 \sin^2(\delta)$$

The angular acceleration of the first pendulum is calculated as:

$$\dot{\omega}_1 = \frac{-\sin(\delta)\left(m_2 L_1 \omega_1^2 \cos(\delta) + m_2 L_2 \omega_2^2\right) - g\left(M \sin(\theta_1) - m_2 \sin(\theta_2)\cos(\delta)\right)}{L_1 \alpha} - \gamma_1 \omega_1$$

The angular acceleration of the second pendulum is calculated as:

$$\dot{\omega}_2 = \frac{\sin(\delta)\left(M L_1 \omega_1^2 + m_2 L_2 \omega_2^2 \cos(\delta)\right) + g\left(M \sin(\theta_1)\cos(\delta) - M \sin(\theta_2)\right)}{L_2 \alpha} - \gamma_2 \omega_2$$

The terms $\gamma_1\omega_1$ and $\gamma_2\omega_2$ are representing the linear damping. Without these terms, the system would approximate an ideal conservative double pendulum. With included damping, energy gradually decreases.

3 Numerical integration method

The simulation uses the classical fourth-order Runge-Kutta method. This method was selected because of its good compromise between accuracy, numerical stability, and computational cost.

For a system of ordinary differential equations

$$\frac{dy}{dt} = f(y, t)$$

the fourth-order Runge-Kutta method calculates four derivative estimates within one time step:

$$k_1 = f(y_n, t_n)$$
$$k_2 = f\left(y_n + \frac{\Delta t}{2} k_1, t_n + \frac{\Delta t}{2}\right)$$
$$k_3 = f\left(y_n + \frac{\Delta t}{2} k_2, t_n + \frac{\Delta t}{2}\right)$$
$$k_4 = f\left(y_n + \Delta t k_3, t_n + \Delta t\right)$$

The next state is then approximated as

$$y_{n+1} = y_n + \frac{\Delta t}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right)$$

Lower-order methods, such as Euler's method or for example RK2, require fewer derivative evaluations per step, but they accumulate larger numerical errors. This is especially problematic for a non-linear and chaotic system like the double pendulum. Higher-order and adaptive methods can provide more accuracy, but they are more complex and less convenient for fixed-step real-time animation. Therefore, RK4 was chosen as the most appropriate method for this project.

3.1 The Choice of Fourth-Order Runge-Kutta

The fourth-order version of the Runge-Kutta method was chosen because it provides a strong balance between accuracy and computational effort. The first-order Euler method evaluates the derivative only once per time step. Although it is very primitive and sufficiently inaccurate for a chaotic nonlinear system. Numerical errors accumulate quickly and lead to unrealistic energy behavior. Second-order methods, such as the midpoint method or RK2, improve the approximation by evaluating the derivative at an intermediate point. However, they still require smaller time steps to reach the same accuracy as RK4. Third-order methods are more accurate than RK2, but RK4 is more widely used in practice due to a noticeable improvement in accuracy while requiring only four derivative evaluations per time step. In this project, the time step is fixed because of real-time visualization. RK4 is well suited for such a structure because it is a one-step method and does not require storing previous states. It can update the state vector directly using the current state and the derivative function. These are the main factors of selecting RK4 as the most practical and sufficient method for this simulation.

4 Implementation Overview

The project was implemented in both Python and MATLAB.

The Python implementation uses:

- NumPy for numerical calculations,
- Matplotlib for real-time visualization,
- LineCollection for the trajectory trace of the second mass.

The MATLAB implementation uses:

- manually implemented RK4 integration,
- real-time updating of plot objects,
- energy visualization,
- normalized angular values for the phase portrait.

Both implementations follow the same computational structure:

1. Define physical parameters and initial conditions.
2. Compute the derivative of the current state.
3. Apply one RK4 integration step.
4. Convert angular coordinates to Cartesian coordinates.
5. Update the pendulum visualization.
6. Calculate and display energy.
7. Update the phase portrait and trajectory trace.

4.1 Comparison of Python and MATLAB Versions

Both versions simulate the same physical system and use the same numerical method. However, they differ in visualization and graphical behavior.

Table 2: Comparison of the Python and MATLAB versions

Criterion	Python version	MATLAB version
Numerical method	RK4	RK4
Physical model	damped double pendulum	damped double pendulum
Visualization library	Matplotlib	MATLAB graphics
Trajectory trace	LineCollection	standard line plot
Energy plot	full history with autoscale	recent time window
Phase portrait	direct θ_1 plot	normalized θ_1 plot
Program loop	infinite loop	closes with figure window

The Python version provides a visually smooth trajectory trace by using Matplotlib's LineCollection. The MATLAB version provides a more stable phase portrait because the angle θ_1 is normalized to the interval $[-\pi, \pi]$. This prevents the graph from drifting when the pendulum completes full rotations. From a physical point of view, both simulations are equivalent because they use the same model and numerical method. From a visualization and analysis point of view, the MATLAB version is more stable for long-term observation, while the Python version provides a more visually polished trajectory trace.

4.2 Limitations

The simulation is a simplified model of a real double pendulum. Several physical effects are neglected:

- mass of the rods,
- air resistance,
- detailed friction in the joints,
- flexibility of the rods,
- three-dimensional motion,
- manufacturing tolerances and joint clearances.

Because of these simplifications, the simulation should be interpreted as an educational and numerical engineering model rather than an exact representation of a real mechanical device.

5 Conclusion

This project demonstrates the numerical simulation of a nonlinear double pendulum using the fourth-order Runge–Kutta method. The implemented model calculates angular velocities and angular accelerations from the current state and integrates them over time to obtain the system motion. The project connects mechanical system modeling, numerical methods, and real-time visualization. The comparison between Python and MATLAB implementations also demonstrates how the same physical model can be implemented and analyzed in different engineering software environments. The simulation confirms that RK4 is a suitable method for this type of system because it provides good accuracy and stability while remaining simple enough for manual implementation and real-time visualization.

Contents

References

[1] D. Baden, *Double Pendulum*, University of Maryland Department of Physics, 2016. Available at: https://physics.umd.edu/hep/drew/pendulum2.html